Assignment 4
Levels of testing - Integration testing and Exploratory testing

Assignment Objectives:

- Exploratory testing (SBTM) and integration testing.
- Use SBMT to test an application.
- Use Mockito for creating mock objects. Create Stubs and Drivers.

Prerequisite:

- Create a team –5 or 6 members.

Assignment = In-Class assignment + Take-Home assignment

- **In-Class assignment**
  - First hour of the laboratory.
  - Maxim 50 XP
  - TO DO:
    1. **Exploratory testing (SBTM)** (20 XP)
       Session Based Test Management – SBTM (See Lecture 6) (50 XP)
       - Conduct a SBTM session. Each student from the team for 20 minutes.
       - Each student will create a different file + the charts and analysis of the session.
       - All students will merge the individual files into one sbtm template file at the end.
       - SBTM template available here (u**se your gmail account**):
       - https://altom.com/version-2-1-of-the-sbtm-session-template-was-released/
       - Instructions how to use the template
       - https://altom.com/sbtm-with-itester-and-google-drive/
    2. **In-class Problem**: Problem and SBTM descriptions are found at the end of this document.
- **Take-Home assignment**
  - At home.
  - Maxim 100 XP
  - TO DO:
    1. **Implementation (Integration testing)** (30 XP)
       - Implementation of a method that establishes if a year is leap or not (*IsLeapYear*).
       - Implementation of a method that computes the number of days between two given dates (that uses the above verification for leap year), *DaysBetween2Dates*.
    2. **Integration testing** (40 XP)
       - Each team will choose a different application from the examples provided in the archive Lab04Files.
       - Simulate the use of the method *DaysBetween2Dates* in the context of the chosen application.
         - Use mocks for *IsLeapYear*.
         - Create driver for *DaysBetween2Dates* using as stub the *IsLearYear*.
       - Use mockito - creating mock objects.
    3. **Simulation of the selected application example – integration testing** (30 XP)
       - Use driver for the *DaysBetween2Dates* in the context of chosen application.
    4. **Take-Home Problem**: Problem and descriptions are found at the end of this document.

Assignment and Delivery date:

1. Assignment date: laboratory 4
2. Delivery date for **In-Class assignment:** laboratory 4 (max 50 XP)
3. Delivery date (first) **for Take-Home assignment**: laboratory 5 (max 100 XP)
4. Delivery date (last) **for Take-Home assignment**: laboratory 6 (max 50XP)

Turn in:

Delivered and presented in class AND upload on Teams (only one student from the team) after Delivery in class the following archive:

a. Lab4_IC_TH_93X_Name01Name02.zip
   i. In-class problem
      1. SBTM files (bugs + charts + analysis)

ii. Take-Home problem
   1. Source code: Implementation for *IsLeapYear*, *DaysBetween2Dates*, simulation of the selected application using *DaysBetween2Dates.*
   2. Mocks for *IsLeapYear*
   3. Driver for *DaysBetween2Dates* using as stub the *IsLeapYear*
   4. Simulation of the selected application using *DaysBetween2Dates*.

iii. document with the list of members. No "guest name"! Every person mentioned as a collaborator must contribute.

References

- See Lecture 3 and 6, Seminar 4
- See documents for DemoLab04_mockito.zip

In-class Lab 4 Assignment

Exploratory testing, Session Based Test Management

Problem: AgeCalculator: The Age Calculator can determine the age or interval between two dates. The calculated age will be displayed in years, months, weeks, days, hours, minutes, and seconds.

https://www.calculator.net/age-calculator.html

Session Charter: Investigate correctness, boundary handling, and robustness of the Age Calculator when computing age from different date inputs.

Test	
Bug	
Setup	
Offchart	
Note	
Next time	
Question	

Take-Home Lab 4 Assignment

Integration testing, stubs and drivers

Problem: Compute the number of days between two given dates, where the computation depends on a **leap year checker**. Application examples where computing the **number of days between two dates** is essential. Each example highlights *why the dependency on correct date computation (including leap years)* is critical in practice:

1) Finance Systems: Loan / Interest Calculation
   A bank needs to compute interest accumulated between two dates.
   Interest is often calculated per day.
   Typical bugs: ignoring leap years and off-by-one day.
2) Software Systems: Subscription / License Validity
   A system checks whether a subscription is still valid.
   Days-between determines expiration and renewal cycles.
   Common bugs: early expiration and late expiration (security risk)
3) Library Borrowing: Return Systems
   A library computes how long a book was borrowed.
   The days between dates determines due dates and the overdue fines.
   Typical bugs: incorrect penalties.
4) Habit Tracking Apps: Fitness
   An app tracks how many days in a row a user maintains a habit (e.g., running).
   The days between dates determines streak length that influences the user motivation and rewards/achievements.
   Typical bugs: incorrect streaks.
5) Delivery Time Estimation Systems
   An e-commerce platform estimates delivery duration.
   The days between dates are used to display delivery estimates and optimize logistics.
   Typical bugs: off-by-one errors and incorrect handling of leap years.